

8장 SQL 응용



340292

필기 25.5, 25.2, 24.2, 23.7, 23.2, 21.8, 21.5, 21.3, 20.6



152 DDL



- DDL(Data Define Language, 데이터 정의어)은 DB 구조, 데이터 형식, 접근 방식 등 DB를 구축하거나 수정할 목적으로 사용하는 언어이다.
- DDL의 3가지 유형

명령어	기능
필기 23.7, 23.2, 21.8, 21.5, ... CREATE	SCHEMA, DOMAIN, TABLE, VIEW, INDEX를 정의함
필기 23.7, 23.2, 21.8, 20.6 ALTER	TABLE에 대한 정의를 변경하는 데 사용함
필기 23.7, 23.2, 21.8, 20.6 DROP	SCHEMA, DOMAIN, TABLE, VIEW, INDEX를 삭제함

340296

20.7



153 CREATE INDEX



CREATE INDEX는 인덱스를 정의하는 명령문이다.

표기 형식

```
CREATE [UNIQUE] INDEX 인덱스명
ON 테이블명(속성명 [ASC | DESC] [,속성명 [ASC | DESC]])
[CLUSTER];
```

- UNIQUE
 - 사용된 경우 : 중복 값이 없는 속성으로 인덱스를 생성함
 - 생략된 경우 : 중복 값을 허용하는 속성으로 인덱스를 생성함
- 정렬 여부 지정
 - ASC : 오름차순 정렬
 - DESC : 내림차순 정렬
 - 생략된 경우 : 오름차순으로 정렬됨

- CLUSTER : 사용하면 인덱스가 클러스터드 인덱스로 설정됨

예제 <고객> 테이블에서 UNIQUE한 특성을 갖는 '고객번호' 속성에 대해 내림차순으로 정렬하여 '고객번호_idx'라는 이름으로 인덱스를 정의하시오.

```
CREATE UNIQUE INDEX 고객번호_idx
ON 고객(고객번호 DESC);
```

340297

20.10, 필기 21.3, 20.9



154 ALTER TABLE



ALTER TABLE은 테이블에 대한 정의를 변경하는 명령문이다.

표기 형식

```
ALTER TABLE 테이블명 ADD 속성명 데이터_타입 [DEFAULT '기본값'];
ALTER TABLE 테이블명 ALTER 속성명 [SET DEFAULT '기본값'];
ALTER TABLE 테이블명 DROP COLUMN 속성명 [CASCADE];
```

- ADD : 새로운 속성(열)을 추가할 때 사용함
- ALTER : 특정 속성의 Default 값을 변경할 때 사용함
- DROP COLUMN : 특정 속성을 삭제할 때 사용함

예제 1 <학생> 테이블에 최대 3문자로 구성되는 '학년' 속성 추가하시오.

```
ALTER TABLE 학생 ADD 학년 VARCHAR(3);
```

예제 2 <학생> 테이블의 '학번' 필드의 데이터 타입과 크기를 VARCHAR(10)으로 하고 NULL 값이 입력되지 않도록 변경하시오.

```
ALTER TABLE 학생 ALTER 학번 VARCHAR(10) NOT NULL;
```

**155 DROP**

DROP은 스키마, 도메인, 기본 테이블, 뷰 테이블, 인덱스, 제약 조건 등을 제거하는 명령문이다.

표기 형식

DROP SCHEMA 스키마명 [CASCADE | RESTRICT];
 DROP DOMAIN 도메인명 [CASCADE | RESTRICT];
 DROP TABLE 테이블명 [CASCADE | RESTRICT];
 DROP VIEW 뷰명 [CASCADE | RESTRICT];
 DROP INDEX 인덱스명 [CASCADE | RESTRICT];
 DROP CONSTRAINT 제약조건명;

- CASCADE : 제거할 요소를 참조하는 다른 모든 개체를 함께 제거함
- RESTRICT : 다른 개체가 제거할 요소를 참조중일 때는 제거를 취소함

예제 <학생> 테이블을 제거하되, <학생> 테이블을 참조하는 모든 데이터를 함께 제거하시오.

DROP TABLE 학생 CASCADE;

**156 DCL**

- DCL(Data Control Language, 데이터 제어어)은 데이터의 보안, 무결성, 회복, 병행 제어 등을 정의하는 데 사용하는 언어이다.
- DCL은 데이터베이스 관리자(DBA)가 데이터를 관리할 목적으로 사용한다.

• DCL의 종류

명령어	기능
필기 20.8 COMMIT	명령에 의해 수행된 결과를 실제 물리적 디스크로 저장하고, 데이터베이스 조작 작업이 정상적으로 완료되었음을 관리자에게 알려줌
필기 21.5, 20.8 ROLLBACK	데이터베이스 조작 작업이 비정상적으로 종료되었을 때 원래의 상태로 복구함
필기 20.8 GRANT	데이터베이스 사용자에게 사용 권한을 부여함
필기 24.7 REVOKE	데이터베이스 사용자의 사용 권한을 취소함

**157 GRANT / REVOKE**

- 데이터베이스 관리자가 데이터베이스 사용자에게 권한을 부여하거나 취소하기 위한 명령어이다.
- GRANT : 권한 부여를 위한 명령어
- REVOKE : 권한 취소를 위한 명령어
- 테이블 및 속성에 대한 권한 부여 및 취소

GRANT 권한_리스트 ON 개체 TO 사용자 [WITH GRANT OPTION];
 REVOKE [GRANT OPTION FOR] 권한_리스트 ON 개체 FROM 사용자 [CASCADE];

- 권한 종류 : ALL, SELECT, INSERT, DELETE, UPDATE 등
- WITH GRANT OPTION : 부여받은 권한을 다른 사용자에게 다시 부여할 수 있는 권한을 부여함
- GRANT OPTION FOR : 다른 사용자에게 권한을 부여할 수 있는 권한을 취소함
- CASCADE : 권한 취소 시 권한을 부여받았던 사용자가 다른 사용자에게 부여한 권한도 연쇄적으로 취소함

예제1 사용자 ID가 "NABI"인 사람에게 <고객> 테이블에 대한 모든 권한과 다른 사람에게 권한을 부여할 수 있는 권한까지 부여하는 SQL문을 작성하시오.

GRANT ALL ON 고객 TO NABI WITH GRANT OPTION;

예제 2 사용자 ID가 “STAR”인 사람에게 부여한 <고객> 테이블에 대한 권한 중 UPDATE 권한을 다른 사람에게 부여할 수 있는 권한만 취소하는 SQL문을 작성하시오.

```
REVOKE GRANT OPTION FOR UPDATE ON 고객 FROM STAR;
```

340301



158 ROLLBACK

20.7, 필기 21.5



- ROLLBACK은 변경되었으나 아직 COMMIT되지 않은 모든 내용들을 취소하고 데이터베이스를 이전 상태로 되돌리는 명령어이다.
- 트랜잭션 전체가 성공적으로 끝나지 못하면 일부 변경된 내용만 데이터베이스에 반영되는 비일관성(Inconsistency) 상태가 될 수 있기 때문에 일부분만 완료된 트랜잭션은 롤백(Rollback)되어야 한다.

340303



159 DML

필기 24.7, 23.5, 20.8, 20.6



- DML(Data Manipulation Language, 데이터 조작어)은 데이터베이스 사용자가 저장된 데이터를 실질적으로 관리하는데 사용되는 언어이다.
- DML은 데이터베이스 사용자와 데이터베이스 관리 시스템 간의 인터페이스를 제공한다.
- DML의 유형

명령어	기능
필기 20.8, 20.6 SELECT	테이블에서 튜플을 검색함
필기 20.8, 20.6 INSERT	테이블에 새로운 튜플을 삽입함
필기 20.8, 20.6 DELETE	테이블에서 튜플을 삭제함
필기 23.5, 20.8, 20.6 UPDATE	테이블에서 튜플의 내용을 갱신함

※ 다음 테이블을 참조하여 핵심 160~162의 예제 결과를 확인하시오.

<사원>

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
임꺽정	인터넷	01/09/69	성산동	80
황진이	편집	07/21/75	연희동	100
김선달	편집	10/22/73	망원동	90
성춘향	기획	02/20/64	망원동	100
장길산	편집	03/11/67	상암동	120
일지매	기획	04/29/78	합정동	110
강호동	인터넷	12/11/80		90

340543



160 삽입문(INSERT INTO~)

24.7, 23.7, 필기 23.7, 23.5



삽입문은 기본 테이블에 새로운 튜플을 삽입할 때 사용한다.

일반 형식

```
INSERT INTO 테이블명([속성명1, 속성명2,...])  
VALUES (데이터1, 데이터2,... );
```

- 대응하는 속성과 데이터는 개수와 데이터 유형이 일치해야 한다.
- 기본 테이블의 모든 속성을 사용할 때는 속성명을 생략할 수 있다.
- SELECT문을 사용하여 다른 테이블의 검색 결과를 삽입할 수 있다.

예제 1 <사원> 테이블에 (이름 – 홍승현, 부서 – 인터넷)을 삽입하시오.

```
INSERT INTO 사원(이름, 부서) VALUES ('홍승현', '인터넷');
```

예제 2 <사원> 테이블에 (장보고, 기획, 05/03/73, 홍제동, 90)을 삽입하시오.

```
INSERT INTO 사원 VALUES ('장보고', '기획',  
#05/03/73#, '홍제동', 90);
```

예제 3 <사원> 테이블에 있는 편집부의 모든 튜플을 편집 부원(이름, 생일, 주소, 기본급) 테이블에 삽입하시오.

```
INSERT INTO 편집부원(이름, 생일, 주소, 기본급)
SELECT 이름, 생일, 주소, 기본급
FROM 사원
WHERE 부서 = '편집';
```

340305



161 삭제문(DELETE FROM~)

(A)

삭제문은 기본 테이블에 있는 튜플들 중에서 특정 튜플(행)을 삭제할 때 사용한다.

일반 형식

```
DELETE FROM 테이블명 [WHERE 조건];
```

- 모든 레코드를 삭제할 때는 WHERE절을 생략한다.
- 모든 레코드를 삭제하더라도 테이블 구조는 남아 있기 때문에 디스크에서 테이블을 완전히 제거하는 DROP과는 다르다.

예제 1 <사원> 테이블에서 “임꺽정”에 대한 튜플을 삭제하시오.

```
DELETE FROM 사원 WHERE 이름 = '임꺽정';
```

예제 2 <사원> 테이블에서 “인터넷” 부서에 대한 모든 튜플을 삭제하시오.

```
DELETE FROM 사원 WHERE 부서 = '인터넷';
```

예제 3 <사원> 테이블의 모든 레코드를 삭제하시오.

```
DELETE FROM 사원;
```

340306



162 갱신문(UPDATE~ SET~)

(A)

갱신문은 기본 테이블에 있는 튜플들 중에서 특정 튜플의 내용을 변경할 때 사용한다.

일반 형식

```
UPDATE 테이블명
SET 속성명 = 데이터[, 속성명=데이터, ...]
[WHERE 조건];
```

예제 1 <사원> 테이블에서 “홍길동”의 “주소”를 “수색동”으로 수정하시오.

```
UPDATE 사원 SET 주소 = '수색동' WHERE 이름 = '홍길동';
```

예제 2 <사원> 테이블에서 “황진이”의 “부서”를 “기획부”로 변경하고 “기본급”을 5만원 인상시키시오.

```
UPDATE 사원
SET 부서 = '기획', 기본급 = 기본급 + 5
WHERE 이름 = '황진이';
```

340307



163 SELECT

(A)

일반 형식

```
SELECT [PREDICATE] [테이블명.]속성명 [AS 별칭][, [테이블명.]속성명, ...]
FROM 테이블명[, 테이블명, ...]
[WHERE 조건]
[GROUP BY 속성명, 속성명, ...]
[HAVING 조건]
[ORDER BY 속성명 [ASC | DESC]];
```

• SELECT절

– PREDICATE : 검색할 튜플 수를 제한하는 명령어를 기술함

▶ DISTINCT : 중복된 튜플이 있으면 그 중 첫 번째 한 개만 표시함

– 속성명 : 검색하여 불러올 속성(열) 또는 속성을 이용한 수식을 지정함

– AS : 속성이나 연산의 이름을 다른 이름으로 표시하기 위해 사용함

• FROM절 : 검색할 데이터가 들어있는 테이블 이름을 기술함

• WHERE절 : 검색할 조건을 기술함

- GROUP BY절 : 특정 속성을 기준으로 그룹화하여 검색할 때 사용한다. 일반적으로 GROUP BY절은 그룹 함수와 함께 사용됨
- HAVING절 : GROUP BY와 함께 사용되며, 그룹에 대한 조건을 지정함
- ORDER BY절 : 데이터를 정렬하여 검색할 때 사용함
 - 속성명 : 정렬의 기준이 되는 속성명을 기술함
 - [ASC | DESC] : 정렬 방식으로, 'ASC'는 오름차순, 'DESC'는 내림차순임. 생략하면 오름차순으로 지정됨

※ 다음 테이블을 참조하여 핵심 164~167의 예제 결과를 확인하시오.

〈사원〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
임꺽정	인터넷	01/09/69	서교동	80
황진이	편집	07/21/75	합정동	100
김선달	편집	10/22/73	망원동	90
성춘향	기획	02/20/64	대흥동	100
장길산	편집	03/11/67	상암동	120
일지매	기획	04/29/78	연남동	110
강건달	인터넷	12/11/80		90

〈여가활동〉

이름	취미	경력
김선달	당구	10
성춘향	나이트댄스	5
일지매	태권	15
임꺽정	씨름	8

340308

24.7, 20.5

164
기본 검색

SELECT 절에 원하는 속성을 지정하여 검색한다.

예제 1 〈사원〉 테이블의 모든 튜플을 검색하시오.

```
SELECT * FROM 사원;
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
임꺽정	인터넷	01/09/69	서교동	80
황진이	편집	07/21/75	합정동	100
김선달	편집	10/22/73	망원동	90
성춘향	기획	02/20/64	대흥동	100
장길산	편집	03/11/67	상암동	120
일지매	기획	04/29/78	연남동	110
강건달	인터넷	12/11/80		90

예제 2 〈사원〉 테이블에서 '주소'만 검색하되 같은 '주소'는 한 번만 출력하시오.

```
SELECT DISTINCT 주소 FROM 사원;
```

〈결과〉

주소
망원동
서교동
합정동
대흥동
상암동
연남동

440245

25.11, 25.4, 22.7, 21.7, 20.7, 필기 25.5, 25.2, 24.7, 23.7, 23.5, 22.3, 21.8

165
조건 지정 검색

- WHERE 절에 다음 연산자들을 이용한 조건을 지정하여 조건에 만족하는 튜플만 검색한다.
- 비교 연산자

연산자	=	<>	>	<	>=	<=
의미	같다	같지 않다	크다	작다	크거나 같다	작거나 같다

- 논리 연산자 : NOT, AND, OR
- LIKE 연산자 : 대표 문자를 이용해 지정된 속성의 값이 문자 패턴과 일치하는 튜플을 검색하기 위해 사용됨

대표 문자	%	_	#
의미	모든 문자를 대표함	문자 하나를 대표함	숫자 하나를 대표함

- IN 연산자 : 필드의 값이 IN 연산자의 수로 지정된 값과 같은 레코드만 검색하며, OR 연산을 수행한 결과와 같음

예제 1 〈사원〉 테이블에서 '기획'부의 모든 튜플을 검색하십시오.

```
SELECT * FROM 사원 WHERE 부서 = '기획';
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
성춘향	기획	02/20/64	대흥동	100
일지매	기획	04/29/78	연남동	110

예제 2 〈사원〉 테이블에서 “기획” 부서에 근무하면서 “대흥동”에 사는 사람의 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 부서 = '기획' AND 주소 = '대흥동';
```

〈결과〉

이름	부서	생일	주소	기본급
성춘향	기획	02/20/64	대흥동	100

예제 3 〈사원〉 테이블에서 성이 '김'인 사람의 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 이름 LIKE "김%";
```

〈결과〉

이름	부서	생일	주소	기본급
김선달	편집	10/22/73	망원동	90

예제 4 〈사원〉 테이블에서 '생일'이 '01/01/69'에서 '12/31/73' 사이인 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 생일 BETWEEN #01/01/69# AND #12/31/73#;
```

〈결과〉

이름	부서	생일	주소	기본급
임꺽정	인터넷	01/09/69	서교동	80
김선달	편집	10/22/73	망원동	90

예제 5 〈사원〉 테이블에서 '주소'가 NULL인 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 주소 IS NULL;
```

〈결과〉

이름	부서	생일	주소	기본급
강건달	인터넷	12/11/80		90

340310



166 정렬 검색



22.5, 21.7, 필기 25.2, 23.5, 22.4

ORDER BY 절에 특정 속성을 지정하여 지정된 속성으로 자료를 정렬하여 검색한다.

예제 〈사원〉 테이블에서 '부서'를 기준으로 오름차순 정렬하고, 같은 '부서'에 대해서는 '이름'을 기준으로 내림차순 정렬시켜서 검색하십시오.

```
SELECT *
FROM 사원
ORDER BY 부서 ASC, 이름 DESC;
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
일지매	기획	04/29/78	연남동	110
성춘향	기획	02/20/64	대흥동	100
임꺽정	인터넷	01/09/69	서교동	80
강건달	인터넷	12/11/80		90
황진이	편집	07/21/75	합정동	100
장길산	편집	03/11/67	상암동	120
김선달	편집	10/22/73	망원동	90

440247
24.10, 24.4, 22.7, 필기 25.8, 24.7, 24.2, 23.5, 22.4, 21.5, 20.9, 20.6

167 하위 질의

A

하위 질의는 조건절에 주어진 질의를 먼저 수행하여 그 검색 결과를 조건절의 피연산자로 사용한다.

예제 1 '취미'가 "나이트댄스"인 사원의 '이름'과 '주소'를 검색하시오.

```
SELECT 이름, 주소
FROM 사원
WHERE 이름 = (SELECT 이름 FROM 여가활동 WHERE 취미 = '나이트댄스');
```

〈결과〉

이름	주소
성춘향	대흥동

예제 2 취미활동을 하지 않는 사원들을 검색하시오.

```
SELECT *
FROM 사원
WHERE 이름 NOT IN (SELECT 이름 FROM 여가활동);
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
황진이	편집	07/21/75	합정동	100
장길산	편집	03/11/67	상암동	120
강건달	인터넷	12/11/80		90

예제 3 "망원동"에 거주하는 사원들의 '기본급'보다 적은 '기본급'을 받는 사원의 정보를 검색하시오.

```
SELECT 이름, 기본급, 주소
FROM 사원
WHERE 기본급 < ALL (SELECT 기본급 FROM 사원 WHERE 주소 = "망원동");
```

〈결과〉

이름	기본급	주소
임꺽정	80	서교동

해설

ALL ()는 하위 질의로 검색된 범위를 기본 질의의 조건으로 사용한다. 즉 〈사원〉 테이블에서 주소가 "망원동"인 사원의 기본급을 모두 추출한 후 추출된 기본급들을 기본 질의의 조건으로 사용한다.

① SELECT 기본급 FROM 사원 WHERE 주소 = "망원동"; : 〈사원〉 테이블에서 '주소'가 "망원동"인 사원들의 '기본급'을 추출한다. 결과는 120, 90이다.

② SELECT 이름, 기본급, 주소 FROM 사원 WHERE 기본급 < ALL (①) : '기본급'이 ①에서 추출된 '기본급'의 모든(ALL) 범위인 120, 90보다 작은(<), 즉 범위 안에서 가장 작은 90보다 작은 기본급을 갖는 자료를 대상으로 '이름', '기본급', '주소'를 출력한다. 결과는 "임꺽정", "80", "서교동"이다.

340312
25.11, 23.4, 22.10, 21.4, 필기 24.5, 20.8

168 그룹 함수

A

그룹 함수는 GROUP BY절에 지정된 그룹별로 속성의 값을 집계할 때 사용된다.

명령어	기능
COUNT(속성명)	그룹별 튜플 수를 구하는 함수
SUM(속성명)	그룹별 합계를 구하는 함수
AVG(속성명)	그룹별 평균을 구하는 함수
MAX(속성명)	그룹별 최대값을 구하는 함수
MIN(속성명)	그룹별 최소값을 구하는 함수
STDDEV(속성명)	그룹별 표준편차를 구하는 함수
VARIANCE(속성명)	그룹별 분산을 구하는 함수



169 그룹 지정 검색



GROUP BY절에 지정한 속성을 기준으로 자료를 그룹화하여 검색한다.

〈상여금〉

부서	이름	상여내역	상여금
기획	홍길동	연장근무	100
기획	일지매	연장근무	100
기획	최준호	야간근무	120
기획	장길산	특별근무	90
인터넷	강건달	특별근무	90
인터넷	서국현	특별근무	90
인터넷	박인식	연장근무	30
편집	김선달	특별근무	80
편집	황종근	연장근무	40
편집	성춘향	야간근무	80
편집	임격정	야간근무	80
편집	황진이	야간근무	50

예제 1 〈상여금〉 테이블에서 '부서'별 '상여금'의 평균을 구하시오.

```
SELECT 부서, AVG(상여금) AS 평균
FROM 상여금
GROUP BY 부서;
```

〈결과〉

부서	평균
기획	102.5
인터넷	70
편집	66

예제 2 〈상여금〉 테이블에서 '상여금'이 100 이상인 사람이 2명 이상인 '부서'의 튜플 수를 구하시오.

```
SELECT 부서, COUNT(*) AS 인원수
FROM 상여금
WHERE 상여금 >= 100
GROUP BY 부서
HAVING COUNT(*) >= 2;
```

〈결과〉

부서	사원수
기획	3



170 집합 연산자를 이용한 통합 질의



집합 연산자를 사용하여 2개 이상의 테이블의 데이터를 하나로 통합한다.

표기 형식

```
SELECT 속성명1, 속성명2, ...
FROM 테이블명
UNION | UNION ALL | INTERSECT | EXCEPT
SELECT 속성명1, 속성명2, ...
FROM 테이블명
[ORDER BY 속성명 [ASC | DESC]];
```

- 두 개의 SELECT문에 기술한 속성들은 개수와 데이터 유형이 서로 동일해야 한다.
- 집합 연산자의 종류(통합 질의의 종류)

집합 연산자	설명	집합 종류
UNION	• 두 SELECT문의 조회 결과를 통합하여 모두 출력함 • 중복된 행은 한 번만 출력함	합집합
UNION ALL	• 두 SELECT문의 조회 결과를 통합하여 모두출력함 • 중복된 행도 그대로 출력함	합집합
INTERSECT	두 SELECT문의 조회 결과 중 공통된 행만 출력함	교집합
EXCEPT	첫 번째 SELECT문의 조회 결과에서 두 번째 SELECT문의 조회 결과를 제외한 행을 출력함	차집합



171 JOIN



- JOIN은 2개의 릴레이션에서 **연관된 튜플들을 결합하여, 하나의 새로운 릴레이션을 반환한다.**
- JOIN은 일반적으로 FROM절에 기술하지만, 릴레이션이 사용되는 곳 어디에나 사용할 수 있다.
- JOIN은 크게 INNER JOIN과 OUTER JOIN으로 구분된다.
 - INNER JOIN : THETA JOIN, EQUI JOIN, NATURAL JOIN, NON-EQUI JOIN
 - OUTER JOIN : LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN
- 조건이 없는 INNER JOIN을 수행하면 CROSS JOIN과 동일한 결과를 얻을 수 있다.
- CROSS JOIN(교차 조인)
 - 교차 조인은 조인하는 두 테이블에 있는 튜플들의 순서쌍을 결과로 반환한다.
 - 교차 조인의 결과로 반환되는 테이블의 행의 수는 두 테이블의 행 수를 곱한 것과 같다.

- WHERE절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1, 테이블명2, ...
WHERE 테이블명1.속성명 = 테이블명2.속성명;
```

- NATURAL JOIN절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 NATURAL JOIN 테이블명2;
```

- JOIN ~ USING절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 JOIN 테이블명2 USING(속성명);
```



172 EQUI JOIN(동등 조인)



- JOIN에 참여하는 두 릴레이션의 속성 값을 비교하여 조건을 만족하는 튜플만 반환하는 조인을 THETA JOIN(세타 조인)이라고 하며, 조인에 사용되는 조건에는 =, ≠, <, ≤, >, ≥가 있다.
- EQUI JOIN(동등 조인)은 조인에 사용되는 조건 중 =(equal) 비교에 의해 같은 값을 가지는 행을 연결하여 결과를 생성하는 방법이다.
- EQUI JOIN의 결과 표시되는 속성(차수)은 두 릴레이션의 속성을 더한 것과 같다.
- EQUI JOIN에서 JOIN 조건이 '='일 때 동일한 속성이 두 번 나타나게 되는데, 이 중 중복된 속성을 제거하여 같은 속성을 한 번만 표기하는 방법을 NATURAL JOIN(자연 조인)이라고 한다.
- EQUI JOIN에서 연결 고리가 되는 공통 속성을 JOIN 속성이라고 한다.

- LEFT OUTER JOIN : INNER JOIN의 결과를 구한 후, 우측 향 릴레이션의 어떤 튜플과도 맞지 않는 좌측 향의 릴레이션에 있는 튜플들에 NULL 값을 붙여서 INNER JOIN의 결과에 추가함

- 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 LEFT OUTER JOIN 테이블명2
ON 테이블명1.속성명 = 테이블명2.속성명;
```

- RIGHT OUTER JOIN : INNER JOIN의 결과를 구한 후, 좌측 향 릴레이션의 어떤 튜플과도 맞지 않는 우측 향의 릴레이션에 있는 튜플들에 NULL 값을 붙여서 INNER JOIN의 결과에 추가함

- 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 RIGHT OUTER JOIN 테이블명2
ON 테이블명1.속성명 = 테이블명2.속성명;
```



173 OUTER JOIN



예제 <학생> 테이블과 <학과> 테이블에서 '학과코드' 값이 같은 튜플을 JOIN하여 '학번', '이름', '학과코드', '학과명'을 출력하는 SQL문을 작성하시오. 이때, '학과코드'가 입력되지 않은 학생도 출력하시오.

```
SELECT 학번, 이름, 학생.학과코드, 학과명
FROM 학생 LEFT OUTER JOIN 학과
ON 학생.학과코드 = 학과.학과코드;
```

해설 INNER JOIN을 하면 '학과코드'가 입력되지 않은 "박치민"은 출력되지 않는다. 그러므로 JOIN 구문을 기준으로 왼쪽 테이블, 즉 <학생>의 자료는 모두 출력되는 LEFT JOIN을 사용한 것이다. 다음과 같이 JOIN 구문을 기준으로 테이블의 위치를 교환하여 RIGHT JOIN을 사용해도 결과는 같다.

```
SELECT 학번, 이름, 학생.학과코드, 학과명
FROM 학과 RIGHT OUTER JOIN 학생
ON 학과.학과코드 = 학생.학과코드;
```

<결과>

학번	이름	학과코드	학과명
15	고길동	com	컴퓨터
16	이순신	han	국어
17	김선달	com	컴퓨터
19	아무개	han	국어
37	박치민		



- 트리거(Trigger)는 데이터베이스 시스템에서 데이터의 삽입(Insert), 갱신(Update), 삭제(Delete) 등의 이벤트(Event)가 발생할 때 관련 작업이 자동으로 수행되게 하는 절차형 SQL이다.
- 트리거는 데이터베이스에 저장되며, 데이터 변경 및 무결성 유지, 로그 메시지 출력 등의 목적으로 사용된다.